

Python Plotting Cheatsheet

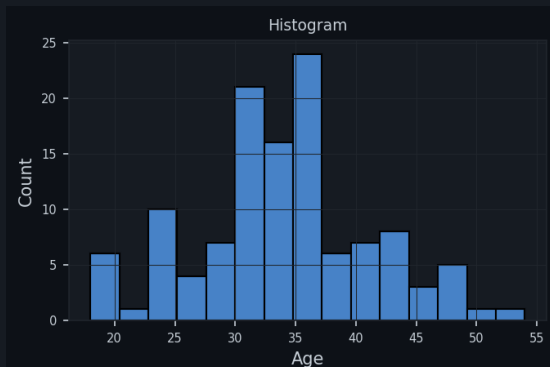
Matplotlib · Seaborn · Plotly — Every chart type with live examples

■ Quick Selection: Count→Countplot | Avg by group→Barplot | Relationship→Scatter | Distribution→Hist/KDE | Outliers→Box/Violin | Correlation→Heatmap | Trend→Line | Interactive/3D/Finance→Plotly

① Distribution Plots [Matplotlib / Seaborn]

■ Histogram

Shows frequency distribution by dividing data into bins.

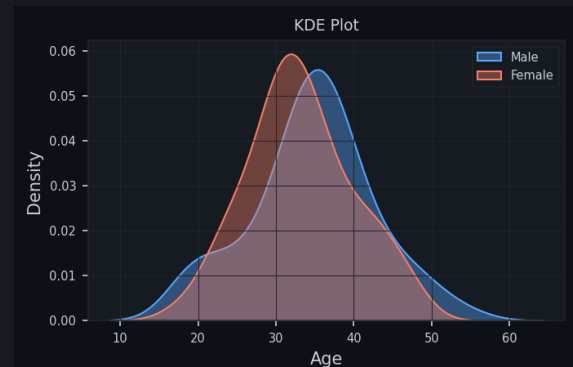


```
sns.histplot(df['Age'], bins=20, kde=False)
```

◆ When to use: Understand shape (skewed / normal) of numeric data.

■ KDE Plot

Smooth continuous probability density — polished histogram alternative.

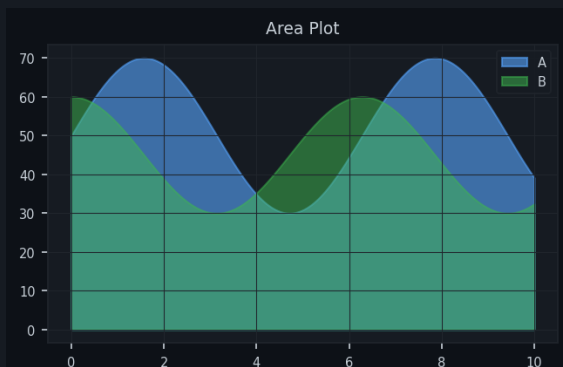


```
sns.kdeplot(df['Age'], fill=True, color='steelblue')
```

◆ When to use: Compare distributions of multiple groups on same axis.

■ Area Plot

Filled line — good for cumulative trends or volume over time.

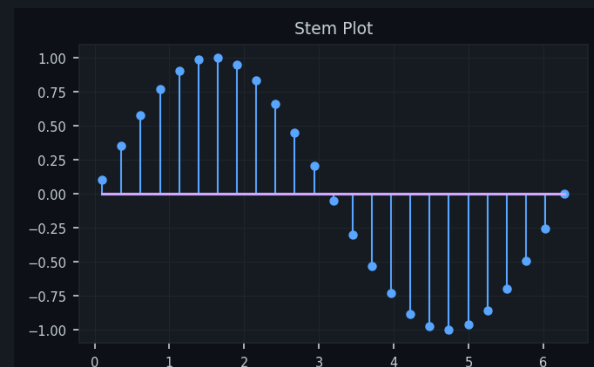


```
ax.fill_between(x, y, alpha=0.5, color='steelblue')
```

◆ When to use: Show volume/magnitude change across continuous axis.

■ Stem Plot

Each value shown with a vertical stem from baseline.



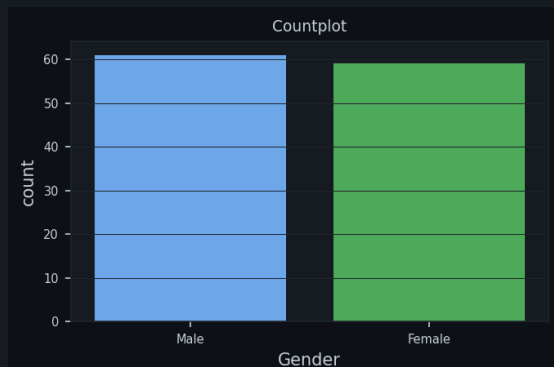
```
ax.stem(x, y, linefmt='C0-', markerfmt='C0o')
```

◆ When to use: Discrete signal values or sparse data points.

② Categorical Plots [Matplotlib / Seaborn]

■ Countplot

Counts occurrences of each category — histogram for categorical data.

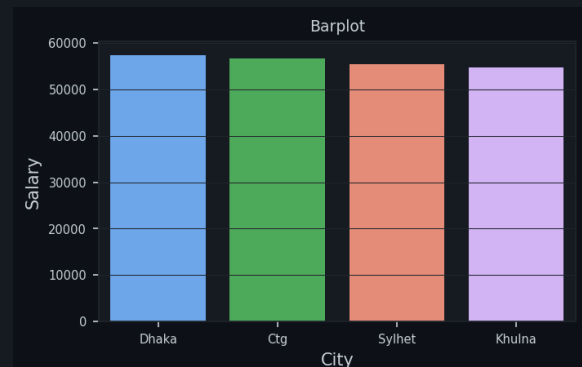


```
sns.countplot(x='Gender', data=df)
```

◆ When to use: First EDA step — see how many rows per category.

■ Barplot

Shows mean (or agg) of a numeric variable per category.

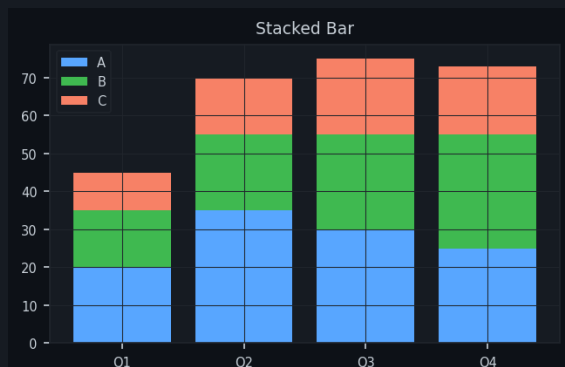


```
sns.barplot(x='City', y='Salary', data=df, errorbar=None)
```

◆ When to use: Compare average values across groups.

■ Stacked Bar

Bars divided into segments showing sub-group composition.

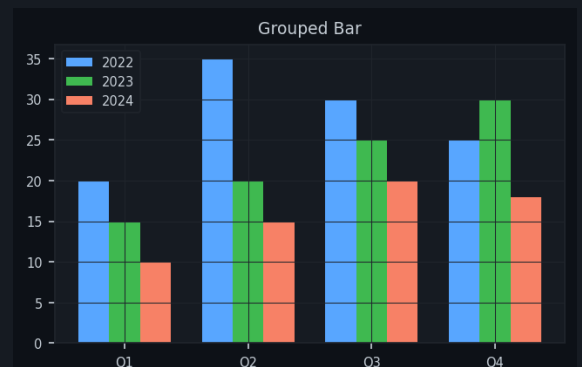


```
ax.bar(cats, a) ax.bar(cats, b, bottom=a)
```

◆ When to use: Part-to-whole relationships across categories.

■ Grouped Bar

Side-by-side bars for multiple sub-groups within each category.

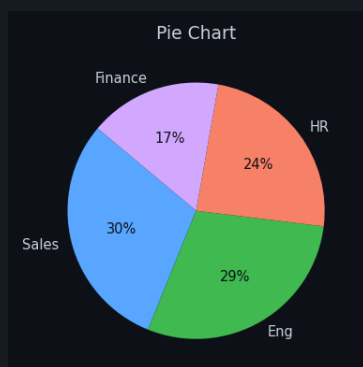


```
ax.bar(x-w, v1, width=w) ax.bar(x+w, v2, width=w)
```

◆ When to use: Direct comparison of multiple sub-groups.

■ Pie Chart

Circular chart showing proportions of a whole. Best for ≤5 categories.

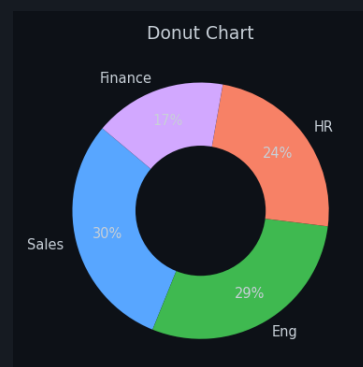


```
ax.pie(sizes, labels=labels, autopct='%1.1f%%')
```

◆ When to use: Percentage share when total = 100%.

■ Donut Chart

Pie with centre cut out — cleaner, allows centre KPI text.



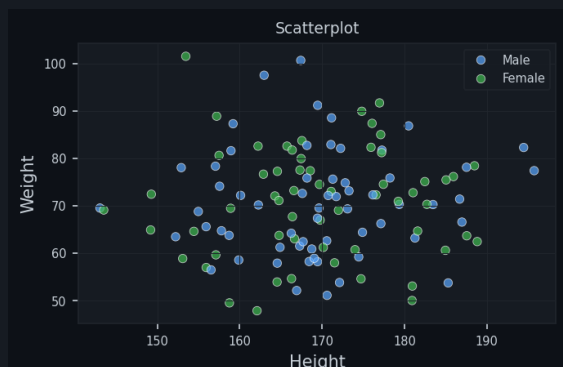
```
ax.pie(sizes, wedgeprops={'width':0.5}, autopct='%1.0f%%')
```

◆ When to use: Modern alternative to pie; add metric in the centre.

③ Relationship & Bivariate [Matplotlib / Seaborn]

■ Scatterplot

Shows relationship between two numeric variables.

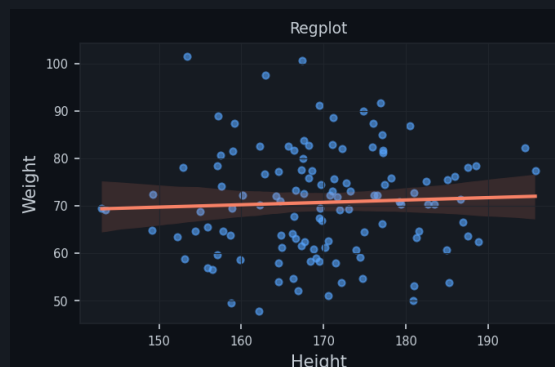


```
sns.scatterplot(x='Height', y='Weight', hue='Gender', data=df)
```

◆ When to use: Find correlations, clusters, outliers between two numeric cols.

■ Regplot (Regression)

Scatter + fitted regression line + confidence interval.

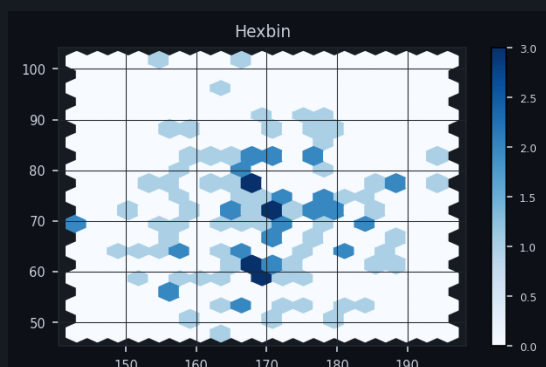


```
sns.regplot(x='Height', y='Weight', data=df)
```

◆ When to use: Check linear relationship; see trend direction and strength.

■ Hexbin

Hex-tile density map — solves over-plotting in large datasets.

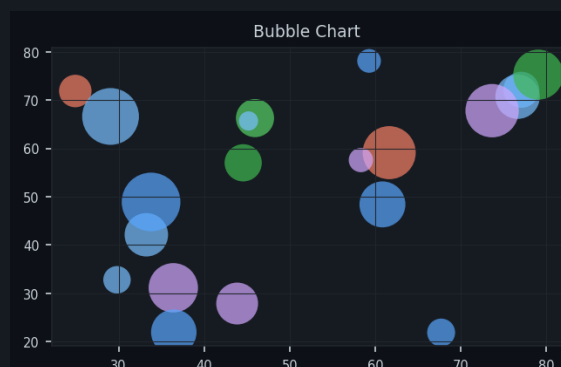


```
ax.hexbin(df['Height'], df['Weight'], gridsize=20, cmap='Blues')
```

◆ When to use: Large datasets (10k+ points) where scatter is too crowded.

■ Bubble Chart

Scatter where a third variable controls point size.



```
ax.scatter(x, y, s=size_var*10, c=color_var, alpha=0.7)
```

◆ When to use: Show 3 numeric dimensions simultaneously.

■ Jointplot

Central scatter + marginal histograms — full bivariate view.



```
sns.jointplot(x='Height', y='Weight', data=df, kind='scatter')
```

◆ When to use: Explore bivariate + univariate distribution in one chart.

■ Pairplot

Grid of scatter plots for all numeric column pairs.



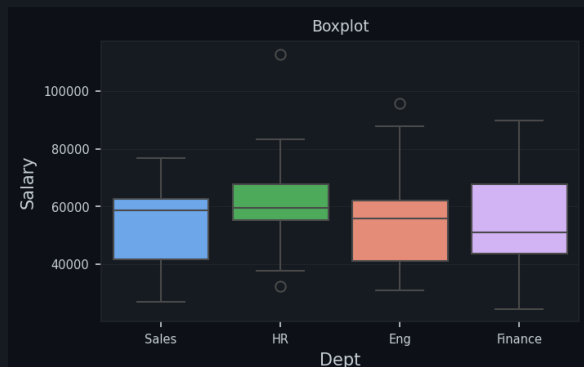
```
sns.pairplot(df[['Age', 'Salary', 'Score']], hue='Gender')
```

◆ When to use: Quick multi-variable EDA — spot all pairwise relationships.

④ Spread & Outlier Plots [Matplotlib / Seaborn]

■ Boxplot

Shows median, IQR, whiskers ($1.5 \times \text{IQR}$), and outlier points.

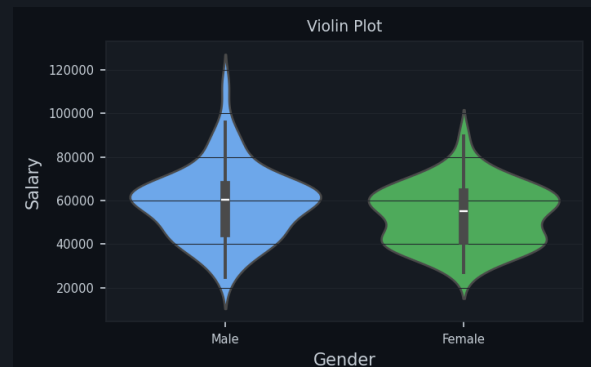


```
sns.boxplot(x='Dept', y='Salary', data=df)
```

◆ When to use: Detect outliers and compare spread across groups.

■ Violin Plot

Boxplot + KDE — shows distribution shape alongside summary stats.



```
sns.violinplot(x='Gender', y='Salary', data=df, inner='box')
```

◆ When to use: When you want both spread and distribution shape.

■ Stripplot

All individual data points scattered along category axis.

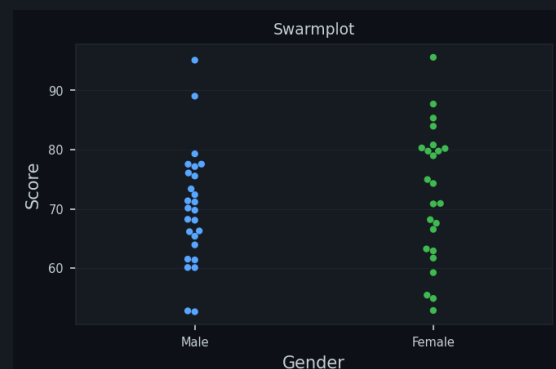


```
sns.stripplot(x='Dept', y='Salary', data=df, jitter=True, size=4)
```

◆ When to use: Small-to-medium datasets where showing every point matters.

■ Swarmplot

Non-overlapping stripplot — arranges points so none overlap.

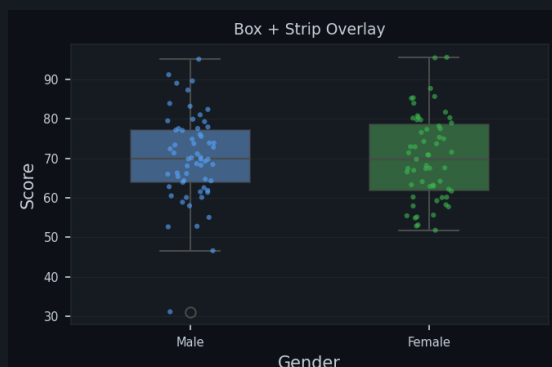


```
sns.swarmplot(x='Gender', y='Score', data=df, size=4)
```

◆ When to use: Better than stripplot for <300 rows.

■ Box + Strip Overlay

Boxplot and stripplot overlaid — summary stats + every data point.

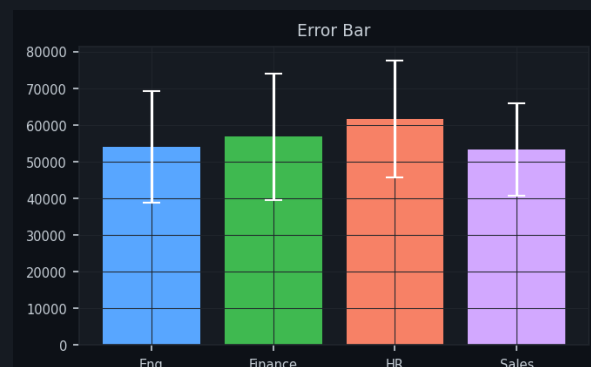


```
sns.boxplot(...) sns.stripplot(..., alpha=0.5)
```

◆ When to use: Best of both worlds for moderate-sized datasets.

■ Error Bar Chart

Bar chart with error bars showing std dev or confidence interval.



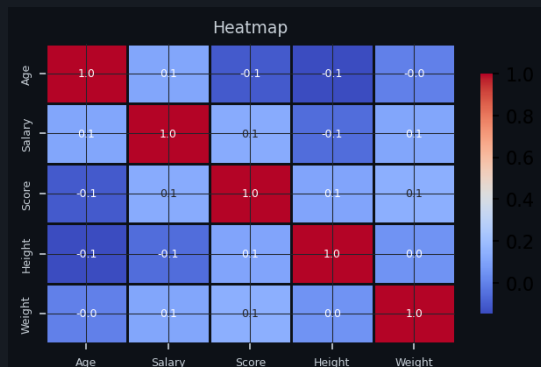
```
ax.bar(labels, means, yerr=stds, capsize=5)
```

◆ When to use: Show uncertainty / variability around mean values.

⑤ Correlation & Matrix [Seaborn]

Heatmap

Colour-encoded matrix showing pairwise correlations.



```
sns.heatmap(df.corr(), annot=True, fmt='.2f',  
cmap='coolwarm')
```

◆ When to use: Find strongly correlated features before modelling.

Clustermap

Heatmap with hierarchical clustering on rows and columns.



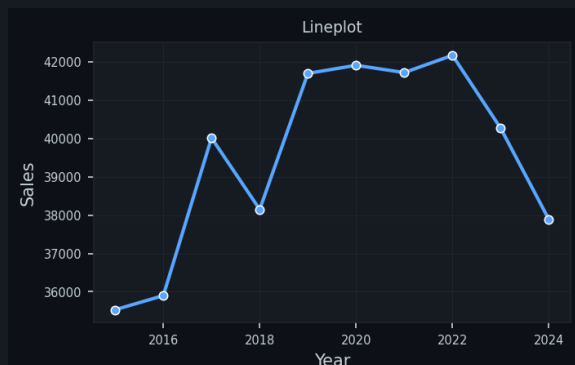
```
sns.clustermap(df[num_cols], cmap='coolwarm',  
standard_scale=1)
```

◆ When to use: Discover natural groupings of similar variables.

⑥ Trend & Time-Series [Matplotlib / Seaborn]

Lineplot

Connects ordered points — ideal for time-series trends.

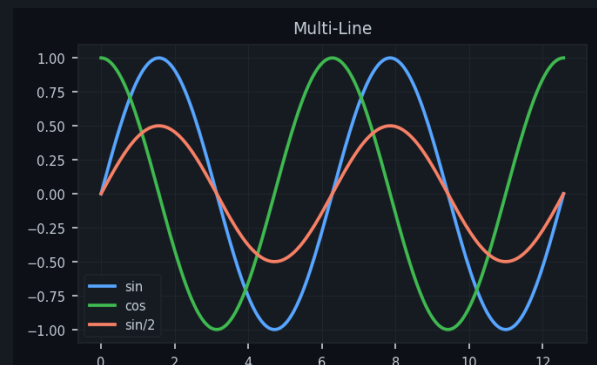


```
sns.lineplot(x='Year', y='Sales', data=df, marker='o')
```

◆ When to use: Show how a metric changes over time.

Multi-Line Plot

Multiple series on one axis for comparison.



```
for col in cols: ax.plot(x, df[col], label=col)
```

◆ When to use: Compare several variables over the same period.

Step Plot

Line that steps horizontally — ideal for discrete/interval data.

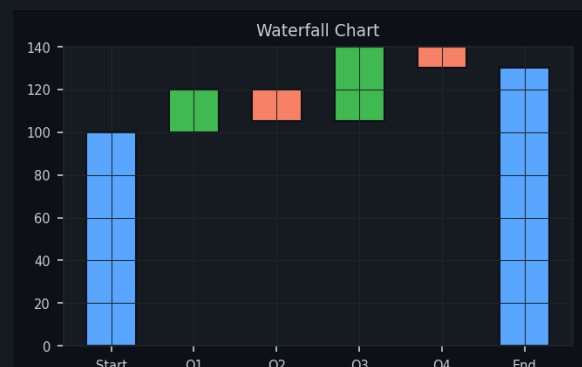


```
ax.step(x, y, where='mid', color='steelblue')
```

◆ When to use: Stock limits, threshold changes, discrete intervals.

Waterfall Chart

Shows positive/negative increments building to a final total.



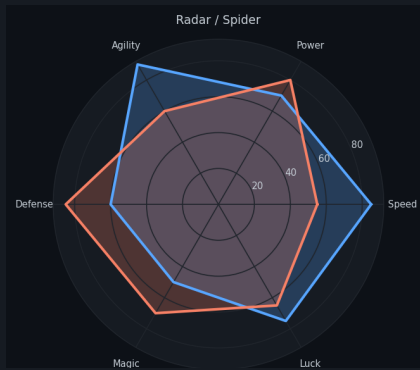
```
ax.bar(labels, deltas, bottom=running_totals,  
color=colors)
```

◆ When to use: Financial P&L, pipeline stages, cumulative-change story.

⑦ Specialty [Matplotlib]

■ Radar / Spider

Multi-axis chart showing values along radial axes from centre.



```
ax = fig.add_subplot(111, polar=True) ax.plot(angles, values) ax.fill(angles, values, alpha=0.2)
```

◆ When to use: Compare multi-dimensional profiles (player stats, product scores).

■ Subplot Grid

Multiple charts arranged in a grid for dashboards.



```
fig, axes = plt.subplots(2, 2, figsize=(10,8)) axes[0,0].hist(...)
```

◆ When to use: Show several related plots side-by-side in one figure.

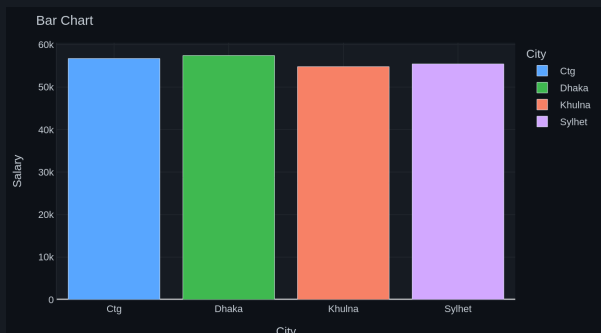
⑧ Plotly — Interactive Charts

■ Plotly charts are interactive in browsers (hover, zoom, pan). Use `px` (`plotly.express`) for quick charts, `go` (`plotly.graph_objects`) for full control. Export to HTML with `fig.write_html()` or PNG with `fig.write_image()`.

```
import plotly.express as px import plotly.graph_objects as go from plotly.subplots import make_subplots # Quick chart (plotly.express) fig = px.scatter(df, x='Height', y='Weight', color='Gender') fig.show() # Save interactive HTML fig.write_html('chart.html') # Save static image (needs kaleido) fig.write_image('chart.png', width=1200, height=600, scale=2)
```

■ px.bar — Bar Chart

Coloured bar chart with hover tooltips and zoom. Supports `facet_col/row` for grid views.

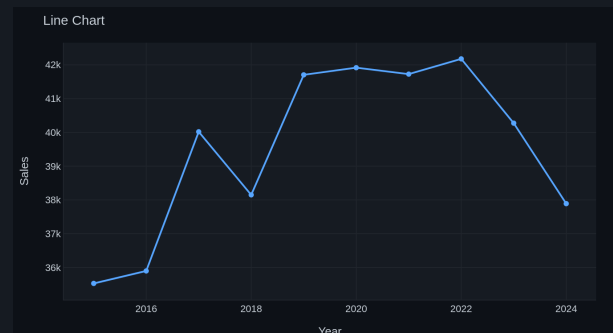


```
fig = px.bar(df, x='City', y='Salary', color='City') fig.show()
```

◆ When to use: Interactive category comparisons; replace static barplot.

■ px.line — Line Chart

Interactive time-series with hover values, range slider, and multi-line support.

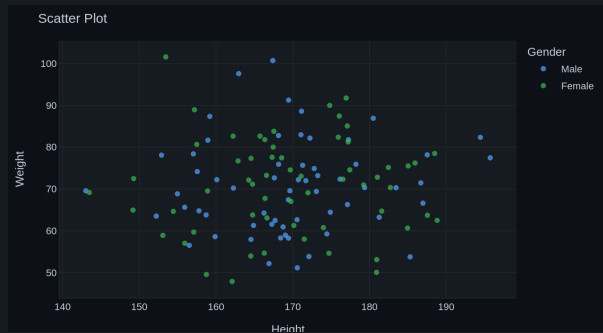


```
fig = px.line(df, x='Year', y='Sales', markers=True) fig.show()
```

◆ When to use: Time-series dashboards; drill-down trend exploration.

■ px.scatter — Scatter Plot

Interactive scatter with hover data, trendline, animation_frame for time-lapse.



```
fig = px.scatter(df, x='H', y='W', color='Gender',  
hover_data=['Age']) fig.show()
```

◆ When to use: Explore clusters interactively; great for presentations.

■ px.histogram — Histogram

Interactive histogram with marginal plots and overlay/stack modes.

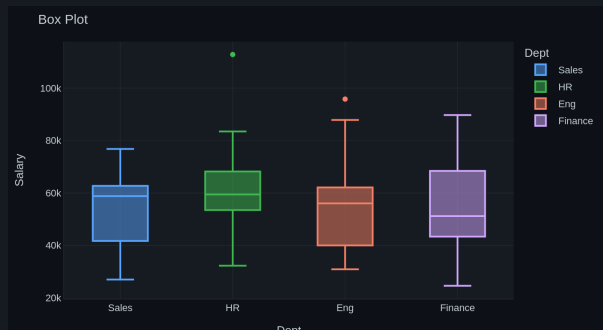


```
fig = px.histogram(df, x='Age', color='Gender',  
barmode='overlay') fig.show()
```

◆ When to use: Quickly compare distributions across groups.

■ px.box — Box Plot

Interactive boxplot with hover showing Q1/Q2/Q3 and outlier values.



```
fig = px.box(df, x='Dept', y='Salary', color='Dept')  
fig.show()
```

◆ When to use: Interactive outlier inspection — click points to identify them.

■ px.violin — Violin Plot

Interactive violin with box overlay and individual point jitter.

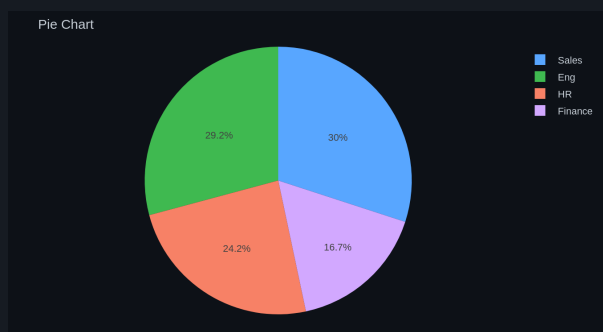


```
fig = px.violin(df, x='Gender', y='Salary', box=True,  
points='outliers') fig.show()
```

◆ When to use: Distribution shape exploration with hover details.

■ px.pie — Pie Chart

Interactive pie with hover % and click-to-explode functionality.

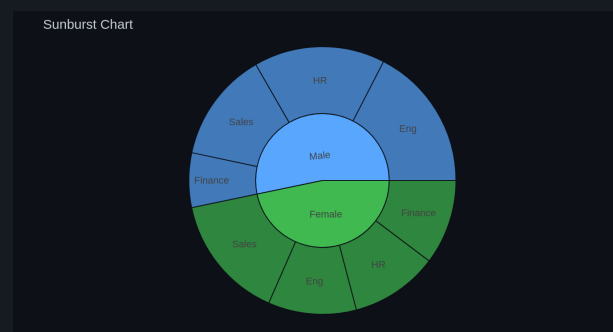


```
fig = px.pie(df, names='Dept', values='Salary')  
fig.show()
```

◆ When to use: Proportion charts in dashboards — hover shows exact values.

■ px.sunburst — Sunburst

Hierarchical pie chart — outer rings show sub-categories.

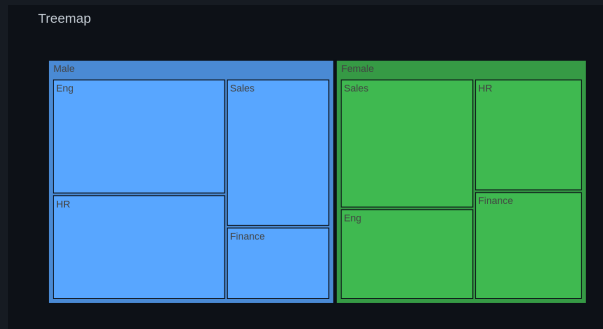


```
fig = px.sunburst(df, path=['Gender', 'Dept'],  
values='Salary') fig.show()
```

◆ When to use: Visualise hierarchical proportions (e.g. category → sub-category).

px.treemap — Treemap

Nested rectangles showing hierarchical data; size = value.

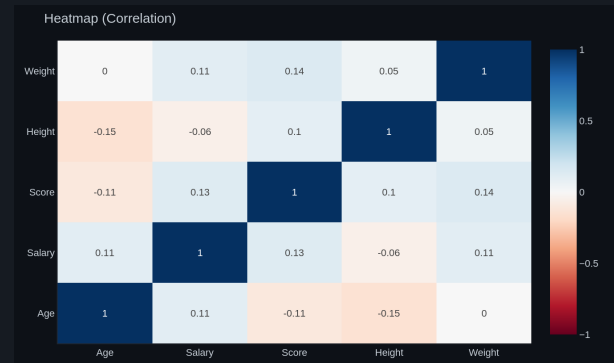


```
fig = px.treemap(df, path=['Gender', 'Dept'],
values='Salary') fig.show()
```

◆ When to use: Hierarchical data where area comparison matters more than angle.

go.Heatmap — Heatmap

Interactive correlation or pivot-table heatmap with hover values.

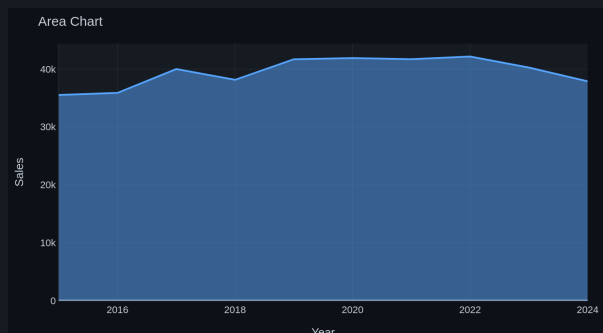


```
fig = go.Figure(go.Heatmap(z=corr.values, x=cols,
y=cols, colorscale='RdBu', texttemplate='%{text}'))
fig.show()
```

◆ When to use: Interactive matrix exploration; click to filter in dashboards.

px.area — Area Chart

Filled line chart; supports stackgroup for stacked areas.

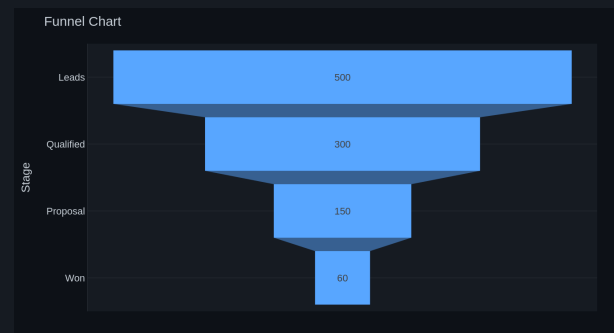


```
fig = px.area(df, x='Year', y='Sales') # Stacked: add
color='Category'
```

◆ When to use: Volume over time; stacked areas for part-to-whole trends.

px.funnel — Funnel Chart

Horizontal bars decreasing in size through pipeline stages.



```
funnel_df = pd.DataFrame({'Stage': [...],
'Count': [...]} ) fig = px.funnel(funnel_df, x='Count',
y='Stage') fig.show()
```

◆ When to use: Sales/marketing pipelines, conversion rate analysis.

px.scatter_3d — 3D Scatter

Fully rotatable 3D scatter — explore three numeric dimensions.



```
fig = px.scatter_3d(df, x='Age', y='Salary',
z='Score', color='Gender') fig.show()
```

◆ When to use: Multi-dimensional data exploration; stunning in presentations.

go.Candlestick — Candlestick

OHLC candlestick chart for financial price data.

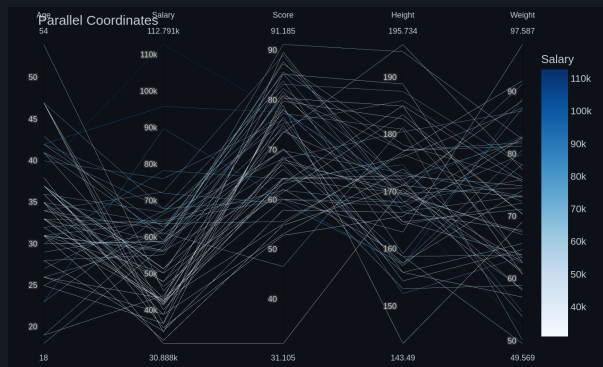


```
fig = go.Figure(go.Candlestick( x=dates, open=o,
high=h, low=l, close=c)) fig.show()
```

◆ When to use: Stock price / financial time-series analysis.

■ px.parallel_coordinates

Multi-axis parallel lines — each line is one data row.

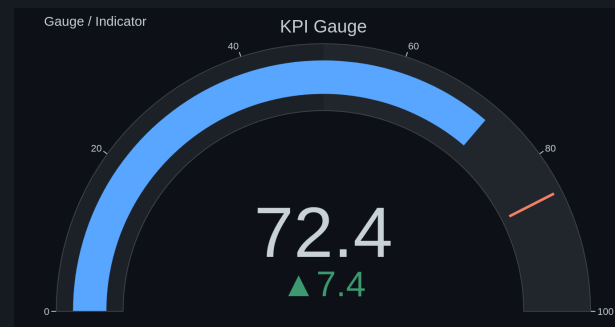


```
fig = px.parallel_coordinates(df, color='Salary',
dimensions=['Age', 'Salary', 'Score']) fig.show()
```

◆ When to use: Explore patterns across many numeric variables simultaneously.

■ go.Indicator — Gauge / KPI

Speedometer-style gauge with delta from target — perfect for dashboards.

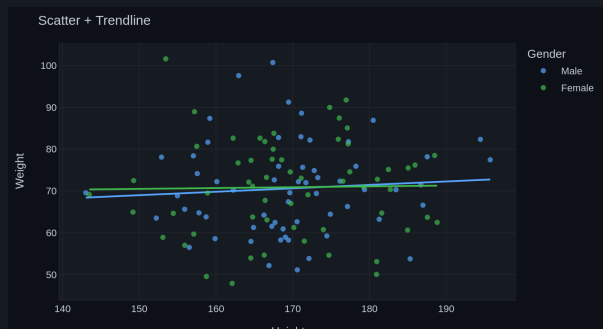


```
fig = go.Figure(go.Indicator(
mode='gauge+number+delta', value=72.4,
delta={'reference':65},
gauge={'axis':{'range':[0,100]}})) fig.show()
```

◆ When to use: KPI dashboards; show current vs target value.

■ px.scatter + trendline

Scatter with OLS/LOWESS trendline computed automatically.

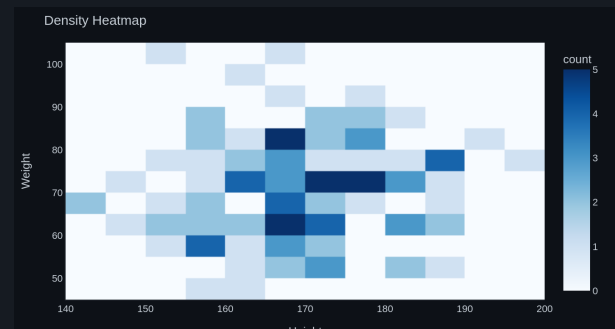


```
fig = px.scatter(df, x='Height', y='Weight',
trendline='ols', color='Gender') fig.show()
```

◆ When to use: Show regression fit interactively; hover to see equation.

■ px.density_heatmap

2D histogram as a heatmap — alternative to hexbin.



```
fig = px.density_heatmap(df, x='Height', y='Weight',
nbinsx=20, nbinsy=20) fig.show()
```

◆ When to use: Large scatter datasets where hexbin is not available.

⑨ Complete Quick-Reference

Plot Type	Category	Use When...
Countplot	Categorical	Count rows per category (sns)
Barplot	Categorical	Avg numeric per category (sns)
Pie / Donut	Categorical	Proportions; ≤5 categories (mpl)
Stacked / Grouped Bar	Categorical	Sub-group comparison (mpl)
px.bar	Categorical	Interactive bar chart (plotly)
px.pie / Sunburst	Categorical	Interactive proportions / hierarchy (plotly)
px.treemap	Categorical	Nested rectangle hierarchy (plotly)
px.funnel	Categorical	Pipeline/conversion stages (plotly)
Histogram	Distribution	Bin frequency; numeric shape (sns)
KDE Plot	Distribution	Smooth density curve (sns)
Area Plot	Distribution	Volume over continuous axis (mpl)
px.histogram	Distribution	Interactive histogram + marginals (plotly)

Scatterplot	Relationship	Correlation between 2 numerics (sns)
Regplot	Relationship	Scatter + regression line (sns)
Hexbin	Relationship	Dense scatter over-plotting fix (mpl)
Bubble	Relationship	Scatter with size = 3rd variable (mpl)
Pairplot	Relationship	All pairwise scatters (sns)
Jointplot	Relationship	Bivariate + marginal dists (sns)
px.scatter	Relationship	Interactive scatter + hover (plotly)
px.scatter_3d	Relationship	Rotatable 3D scatter (plotly)
px.scatter+trendline	Relationship	Scatter with OLS/LOWESS fit (plotly)
px.density_heatmap	Relationship	2D histogram heatmap (plotly)
px.parallel_coords	Relationship	Multi-axis exploration (plotly)
Boxplot	Spread	Median, IQR, outliers (sns)
Violin	Spread	Box + KDE shape (sns)
Strip / Swarm	Spread	Every point visible (sns)
Error Bar	Spread	Mean ± std / CI (mpl)
px.box / px.violin	Spread	Interactive outlier inspection (plotly)
Heatmap	Correlation	Colour-coded correlation matrix (sns)
Clustermap	Correlation	Heatmap + hierarchical clustering (sns)
go.Heatmap	Correlation	Interactive matrix (plotly)
Lineplot	Trend	Time series; ordered x-axis (sns)
Multi-Line	Trend	Compare multiple time-series (mpl)
Step Plot	Trend	Discrete / step-function data (mpl)
Waterfall	Trend	Cumulative increments; P&L (mpl)
px.line / px.area	Trend	Interactive time-series (plotly)
go.Candlestick	Trend	OHLC financial prices (plotly)
Radar / Spider	Specialty	Multi-attribute profiles (mpl)
Stem Plot	Specialty	Sparse discrete signals (mpl)
go.Indicator/Gauge	Specialty	KPI / dashboard gauge (plotly)
Subplot Grid	Layout	Dashboard: multi-plot figure (mpl)

⑩ Import & Setup Boilerplate

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go

# Matplotlib / Seaborn theme
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_theme(style='darkgrid', palette='deep')

# Plotly theme
import plotly.io as pio
pio.templates.default = 'plotly_dark'

# Save a matplotlib plot
plt.savefig('plot.png', dpi=300, bbox_inches='tight')

# Save a plotly chart
fig.write_html('chart.html')

# interactive
fig.write_image('chart.png', scale=2)

# static PNG
```

■ Pro Tips: (1) Use plotly.express for 90% of charts — it's one-liners. (2) Add fig.update_layout(template='plotly_dark') once. (3) sns.set_theme() styles all seaborn plots globally. (4) plt.tight_layout() prevents clipped labels. (5) Use make_subplots() in Plotly for multi-panel dashboards.

